

Marketing Campaign Revenue Prediction



PROJECT
JENNIFER MARY M

3

Marketing Campaign Revenue Prediction

Project Documentation

1. Project Overview

Marketing Campaign Revenue Prediction is a Streamlit-based machine learning application that predicts campaign revenue and classifies a marketing campaign as PROFIT or LOSS. It accepts campaign performance and marketing details, preprocesses the inputs using saved encoding and scaling objects, and displays prediction results with a campaign visualization.

2. Technologies Used

- Python
- Streamlit
- Pandas
- Joblib
- Machine Learning Models

3. Machine Learning Workflow

1. Load the saved revenue prediction model and scaler.
2. Load the saved profit/loss classification model and scaler.
3. Load the MultiLabelBinarizer and training column information.
4. Collect campaign inputs from the Streamlit interface.
5. Apply one-hot encoding to categorical fields.
6. Encode the marketing channel using the MultiLabelBinarizer.
7. Combine and align the processed features with the training columns.
8. Scale the input data.
9. Predict campaign revenue.
10. Predict PROFIT or LOSS.
11. Display the results and campaign input visualization.

```
Name
x2 nykaa_campaign_data_with_nulls.csv
x2 purple_campaign_data_with_nulls.csv
x2 tira_campaign_data_with_nulls.csv
```

```
[2]: import pandas as pd
import numpy as np

[4]: tira=pd.read_csv("../Dataset/tira_campaign_data_with_nulls.csv")
purple=pd.read_csv("../Dataset/purple_campaign_data_with_nulls.csv")
nykaa=pd.read_csv("../Dataset/nykaa_campaign_data_with_nulls.csv")

[5]: nykaa.head()

[5]:
```

	Campaign_ID	Campaign_Type	Target_Audience	Duration	Channel_Used	Impressions
0	NY-CMP-1000	Social Media	College Students	21.0	WhatsApp, YouTube	8224

```
[17]: df = pd.concat([tira,purple,nykaa],ignore_index=True)

[18]: df.shape

[18]: (166665, 17)

[ ]:
```

```
[22]: Index(['Campaign_ID', 'Campaign_Type', 'Target_Audience', 'Duration',
'Channel_Used', 'Impressions', 'Clicks', 'Leads', 'Conversions',
'Revenue', 'Acquisition_Cost', 'ROI', 'Language', 'Engagement_Score',
'Customer_Segment', 'Date', 'Brand'],
dtype='object')

[23]: df.isnull().sum()

[23]:
```

Campaign_ID	5417
Campaign_Type	8358
Target_Audience	8172
Duration	8323
Channel_Used	8282
Impressions	8224
Clicks	8224
Leads	8090
Conversions	8000
Revenue	8304
Acquisition_Cost	8161
ROI	8298
Language	8299
Engagement_Score	8081
Customer_Segment	8175
Date	8099
Brand	0

```
dtype: int64
```

```
[28]: df.isnull().sum()
```

```
[28]: Campaign_ID      0
      Campaign_Type    0
      Target_Audience  0
      Duration         0
      Channel_Used     0
      Impressions      0
      Clicks           0
      Leads           0
      Conversions      0
      Revenue          0
      Acquisition_Cost  0
      ROI              0
      Language         0
      Engagement_Score  0
      Customer_Segment  0
      Date            8099
      Brand            0
      dtype: int64
```

```
[41]: df["Brand"].value_counts()
```

```
[41]: Brand
      Tira      55555
      Purp1le   55555
      Nykaa     55555
      Name: count, dtype: int64
```

```
[ ]:
```

4. Features

- Accepts duration, impressions, clicks, leads, conversions, acquisition cost, and engagement score.
- Provides selections for campaign type, target audience, language, customer segment, brand, and marketing channel.
- Predicts campaign revenue using a saved regression model.
- Classifies the campaign as PROFIT or LOSS using a saved classification model.
- Uses one-hot encoding and MultiLabelBinarizer for categorical inputs.
- Displays processed input and scaled input.
- Displays predicted revenue in Indian Rupees.
- Shows a bar chart for impressions, clicks, leads, and conversions.

Exploratory Data Analysis (EDA)

```
58]: # Analyze campaign performance across brands

59]: brand_revenue = df.groupby("Brand")["Revenue"].sum()

brand_revenue

59]: Brand
Nykaa      2.820715e+10
Purplle    2.810384e+10
Tira       2.803919e+10
Name: Revenue, dtype: float64
```

Feature Engineering

```
5]: # Profit/Loss flag based on ROI

3]: df["Profit_Flag"] = (df["ROI"] > 0).astype(int)

1]: df["Profit_Flag"].value_counts()

1]: Profit_Flag
1      128774
0       37891
Name: count, dtype: int64
```

```
: mlb = MultiLabelBinarizer()

channel_encoded = pd.DataFrame(
    mlb.fit_transform(df["Channel_Used"]),
    columns=mlb.classes_,
    index=df.index
)

: channel_encoded.head()
```

```
:   Email  Facebook  Google  Instagram  WhatsApp  YouTube
0      1         0      0         0         0         0
1      0         0      0         0         0         1
2      1         0      0         0         0         0
3      0         0      1         0         0         0
4      1         1      0         0         1         0
```

```
]: print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)

MAE: 169586.14374761505
MSE: 68219909616.90498
RMSE: 261189.41329407855
R2 Score: 0.6975405300985511
```

```
dt_mae = mean_absolute_error(
    y_test_reg,
    y_pred_dt
)

dt_mse = mean_squared_error(
    y_test_reg,
    y_pred_dt
)

dt_rmse = dt_mse ** 0.5

dt_r2 = r2_score(
    y_test_reg,
    y_pred_dt
)

print("MAE:", dt_mae)
print("RMSE:", dt_rmse)
print("R2 Score:", dt_r2)
```

```
MAE: 221941.62764
RMSE: 356458.96525813447
R2 Score: 0.43665410876675537
```

```
218]: # Step 4: Evaluate

219]: rf_mae = mean_absolute_error(
        y_test_reg,
        y_pred_rf
    )

    rf_mse = mean_squared_error(
        y_test_reg,
        y_pred_rf
    )

    rf_rmse = rf_mse ** 0.5

    rf_r2 = r2_score(
        y_test_reg,
        y_pred_rf
    )

    print("MAE:", rf_mae)
    print("RMSE:", rf_rmse)
    print("R2 Score:", rf_r2)

MAE: 165811.5693712
RMSE: 253310.96573504488
R2 Score: 0.715511950125379
```

	Model	MAE	RMSE	R2 Score
0	Linear Regression	169586.143748	261189.413294	0.697541
1	Decision Tree Regressor	221941.627640	356458.965258	0.436654
2	Random Forest Regressor	165811.569371	253310.965735	0.715512

5. Input Parameters

Duration: Campaign duration.

Impressions: Number of impressions.

Clicks: Number of clicks.

Leads: Number of leads generated.

Conversions: Number of conversions.

Acquisition Cost: Campaign acquisition cost.

Engagement Score: Campaign engagement score.

Campaign Type: Influencer, Paid Ads, SEO, or Email.

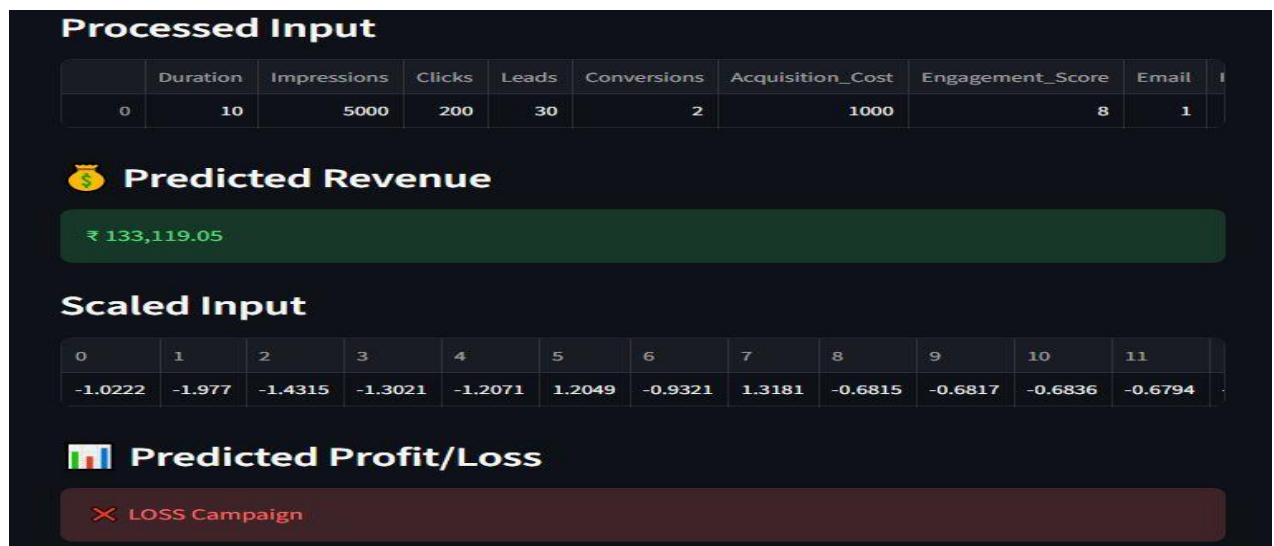
Target Audience: Men, Women, or Youth.

Language: English, Tamil, or Hindi.

Customer Segment: Premium or Regular.

Brand: Purple or Tira.

Marketing Channel: Selected from the saved MultiLabelBinarizer classes.



6. Required Model Files

- revenue_prediction_model.pkl
- revenue_scaler.pkl
- profit_prediction_model.pkl
- classification_scaler.pkl
- mlb.pkl
- training_columns.pkl

7. Project Structure

app.py – Main Streamlit application.

revenue_prediction_model.pkl – Revenue regression model.

revenue_scaler.pkl – Revenue prediction scaler.

profit_prediction_model.pkl – Profit/loss classification model.

classification_scaler.pkl – Classification scaler.

mlb.pkl – MultiLabelBinarizer for marketing channel encoding.

training_columns.pkl – Saved training feature columns.

8. How to Run

12. Install Python.
13. Place app.py and all required .pkl files in the same project folder.
14. Install the required Python packages.
15. Open a terminal in the project folder.
16. Run: streamlit run app.py
17. Open the Streamlit URL displayed in the terminal.

9. Outputs

- Processed campaign input table.
- Predicted revenue in ₹.
- Predicted PROFIT or LOSS classification.
- Bar chart showing impressions, clicks, leads, and conversions.

10. Screenshots

Added screenshots of the input form, predicted revenue, PROFIT/LOSS result, processed input, and campaign input visualization.

11. GitHub Upload

Uploaded app.py and all required .pkl model files to the GitHub repository. A README.md file has been added to describe the project and provide running instructions.